M3 验收报告（Markdown）

Member C：Query / Table / Transaction Integration

对照范围（文件）：`query.py`、`table.py`、`transaction.py`、`lock_manager.py`、`transaction_worker.py`。

---

0. 结论概览

0.1 结构 / 接口

- ☑ **读操作接口支持 `txn=`**：`Query.select(..., txn=None)`、`Query.sum(..., txn=None)` 均存在，并在读路径使用锁。
- ⚠ **写操作接口不支持 `txn=`**：`insert/update/delete/increment` 不接受 `txn` 参数，写路径锁并非在 Query 内实现。
- ☑ **并发控制闭环存在**，但**实现落点偏离计划**：写锁主要由 `transaction.py` 统一规划并获取，而不是按计划在 Query 写路径直接 acquire X。

结构结论：☑ *通过*

0.2 功能 / 行为

- ☑ **strict 2PL**：锁在 `commit/abort` 时统一释放（中途不提前释放）。
- ☑ **no-wait**：锁冲突会抛出冲突异常/失败，事务捕获后 abort。
- ☑ **worker 重试**：abort 后会在 worker 中重试 transaction。
- ☑ **读路径 S 锁接入**：select/sum 逐 rid 读前加 S 锁，失败返回 False，事务会 abort。
- ⚠ **集成级风险**：Table 内部多个共享结构（如 page_directory / key2rid / deleted flags）本身不是全域线程安全，需要依赖上层"记录锁覆盖访问路径"。

功能结论：☑ *事务/锁/重试闭环可跑通；但需在验收中明确"写锁落点在 Transaction，不在 Query"。*

---

1. 需求映射（Plan → 代码落点 → 判定）

需求点	期望行为	代码落点（示例）	判定	备注
读操作拿 S 锁	select/sum 读前 acquire S；冲突 fail	`query.py` 中 select/sum + shared lock helper	☑	失败返回 False
写操作拿 X 锁	insert/update/delete acquire X；冲突 fail	`transaction.py` 统一规划 + `lock_manager.py`	⚠	落点不在 Query
strict 2PL	commit/abort 才 release_all	`transaction.py` commit/abort	☑	中途不释放

需求点	期望行为	代码落点（示例）	判定	备注
no-wait	冲突不等待，直接失败/abort	`lock_manager.py`	☑	抛冲突
失败传播 abort	拿锁失败/返回 False → abort	`transaction.py` run + abort	☑	读路径 False 会 abort
worker 重试	abort 后自动重试直到成功/停止	`transaction_worker.py`	☑	有 retry loop
二级索引加速	select/sum 有 index 用 index	`query.py` select/sum	☑	locate/locate_range
merge 集成	写后触发/应用 merge	`query.py` piggyback + `table.py` merge	☑	apply_merge_if_ready

2. 结构层验收（接口/参数/调用约定）

2.1 Query 必须接口（核心）

- ☑ `select(key, column, query_columns, txn=None)` 存在。
- ☑ `sum(start_range, end_range, aggregate_column_index, txn=None)` 存在。
- ☑ `insert/update/delete/increment` 根据 transaction 调用约定必须"写操作不传 txn"（否则接口不匹配）。

2.2 锁接入职责划分（需要在验收中写清楚）

- ☑ **读锁在 Query 内**：select/sum 会在读前逐 rid 拿 S。
- ☑ **写锁在 Transaction 内**：transaction 会为 insert / update / delete / increment 规划并获取 X 锁。

---

3. 功能层验收（锁/事务/并发）

3.1 读路径：S 锁 + 失败传播

- ☑ `select`：scan / index 返回 rid 后，读前逐 rid acquire S；失败直接 `return False`。
- ☑ `sum`：range/index 或 scan 逐 rid acquire S；失败 `return False`。
- ☑ Transaction：检测到操作失败（False/异常）会 abort，并在 strict 2PL 下统一释放锁。

3.2 写路径：X 锁（落在 Transaction）

- ☑ insert：先对 `("PK", pk)` 进行加锁防并发插入同主键；插入后可补锁 base_rid。
- ☑ update/delete/increment：对目标 `base_rid` 加 X 锁。
- ⚠ Query 写方法本身没有锁逻辑：若有人绕过 Transaction 直接并发调用 Query 写路径，会缺乏锁保护（属于集成风险）。

3.3 strict 2PL / no-wait / retry

- ☑ strict 2PL：`commit/abort` 才 release_all。

- ☑ no-wait：锁冲突直接抛冲突（不等待）。
- ☑ worker 重试：事务 abort 后会循环重试执行。

---

4. Blocker / 风险点

4.1 【偏差】写锁未在 Query 内实现

现状：Query 只对读路径做锁接入；写路径锁由 Transaction 统一规划与获取。

4.2 【集成级风险】Table 共享结构并发安全依赖"记录锁覆盖"

`table.py` 中存在多个共享结构（例如 page_directory / key2rid / deleted flags 等），自身并非全域线程安全；其安全性依赖上层记录锁确保访问路径互斥。

---

5. Table 侧与 Member C 相关的补充点

5.1 merge 集成

- ☑ Table 存在 merge 管线与 apply 逻辑；Query 路径中也有 piggyback apply merge 的调用点。

5.2 metadata persistence（若 M3 有验收点）

- ☑ Table 提供 metadata 的序列化/反序列化接口，并覆盖关键元数据结构。

---

6. 最终判定

6.1 结构判定

- ☑ 读路径 txn 接入完整；
- ☑ 写路径 txn/锁接入通过 transaction

结构结论：☑ *通过*

6.2 功能判定

- ☑ strict 2PL / no-wait / abort / retry 闭环完整；
- ⚠ Table 内部结构线程安全依赖"记录锁覆盖"属于集成级风险。

功能结论：☑ *通过*

---

7. 改进建议

1. 明确团队约定 + README：并发写必须通过 Transaction 调用 Query，禁止直接并发调用 Query 写接口；
2. 对 Table 的关键共享结构写入点考虑轻量锁或在注释中明确依赖"记录锁覆盖"的前提条件。